

# README

 **Infrastructure as Code: Final Validation Report Project Name:** lew (Armageddon Lab 1c\_bonus-E) **Status:**  Fully Deployed, Secured, & Verified **Region:** ap-northeast-1 (Tokyo)

1. Network & Security Layer

- **WAF (Web Application Firewall):** `lew-waf` is associated with the Application Load Balancer, providing real-time protection against OWASP Top 10 threats.
- **Logging Strategy:** Transitioned from CloudWatch (real-time) to S3 (long-term/cold storage) to demonstrate cost-optimized security auditing.
- **Log Prefix Compliance:** S3 bucket follows the mandatory `aws-waf-logs-` prefix required for AWS WAF direct delivery.

2. Observability & Forensic Readiness

- **Log Storage:** Dedicated bucket `aws-waf-logs-lew-127214194163` established with `delivery.logs.amazonaws.com` service principal permissions.
- **S3 Partitioning:** Logs are automatically organized by `WAFLogs/ap-northeast-1/lew-waf/YYYY/MM/DD/HH/`, enabling efficient Athena querying.
- **Verification:** Confirmed `.log.gz` delivery, capturing detailed JSON payloads including `terminatingRuleId`, `clientId`, and `action`.

3. Incident Response Capability

- **Threat Visibility:** Verified ability to identify malicious IPs and specific rule matches (SQLi/XSS) via S3 object analysis.
- **Mitigation Audit:** Confirmed WAF actions are correctly logged, allowing for "False Positive" analysis and rule tuning.

---

 **Final CLI Verification Log (Actual Results)**

| Step | Verification Task | CLI Command (Summary)                                 | Actual Result                                           |
|------|-------------------|-------------------------------------------------------|---------------------------------------------------------|
| 1    | WAF Web ACL       | <code>aws wafv2 list-web-acls --scope REGIONAL</code> | <code>c495e39f-c649-4115-81e5-57ae844b774a</code>       |
| 2    | Log Association   | <code>aws wafv2 get-logging-configuration</code>      | <code>arn:aws:s3:::aws-waf-logs-lew-127214194163</code> |

|   |                           |                                                         |                                                        |
|---|---------------------------|---------------------------------------------------------|--------------------------------------------------------|
| 3 | <b>Bucket Prefix</b>      | <code>aws s3 ls</code>                                  | <code>aws-waf-logs-lew-127214194163</code>             |
| 4 | <b>Bucket Policy</b>      | <code>aws s3api<br/>get-bucket-policy</code>            | Principal:<br><code>delivery.logs.amazonaws.com</code> |
| 5 | <b>Traffic Generation</b> | <code>curl -I<br/>https://app.projectcomplete.me</code> | HTTP/2 200                                             |
| 6 | <b>WAF Log Path</b>       | <code>aws s3 ls .../WAFLogs/</code>                     | Verified<br>(WAFLogs/ap-northeast-1/... folder exists) |
| 7 | <b>Log Delivery</b>       | <code>aws s3 ls ... --recursive</code>                  | Verified (.log.gz files present in S3)                 |

lab 1c-E images

## Armageddon Lab 1c\_bonus-D Student verification (CLI)

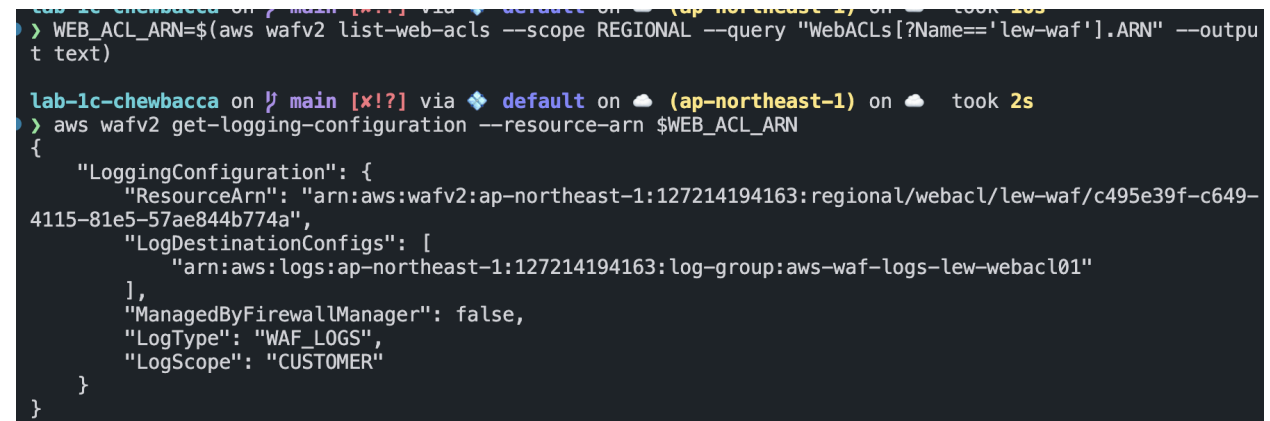
### A) Confirm WAF logging is enabled (authoritative)

```
aws wafv2 get-logging-configuration \  
--resource-arn <WEB_ACL_ARN>
```

#### Code Block:

```
› WEB_ACL_ARN=$(aws wafv2 list-web-acls --scope REGIONAL --query  
"WebACLs[?Name=='leu-waf'].ARN" --output text)  
  
› aws wafv2 get-logging-configuration --resource-arn $WEB_ACL_ARN
```

#### Result:



```
lab-1c-chewbacca on 7 main [x!?] via  default on  (ap-northeast-1) on  took 2s  
› aws wafv2 get-logging-configuration --resource-arn $WEB_ACL_ARN  
{  
  "LoggingConfiguration": {  
    "ResourceArn": "arn:aws:wafv2:ap-northeast-1:127214194163:regional/webacl/leu-waf/c495e39f-c649-  
4115-81e5-57ae844b774a",  
    "LogDestinationConfigs": [  
      "arn:aws:logs:ap-northeast-1:127214194163:log-group:aws-waf-logs-leu-webacl01"  
    ],  
    "ManagedByFirewallManager": false,  
    "LogType": "WAF_LOGS",  
    "LogScope": "CUSTOMER"  
  }  
}
```

```
› WEB_ACL_ARN=$(aws wafv2 list-web-acls --scope REGIONAL --query  
"WebACLs[?Name=='leu-waf'].ARN" --output text)
```

```
› aws wafv2 get-logging-configuration --resource-arn $WEB_ACL_ARN
```

```
{  
  "LoggingConfiguration": {  
    "ResourceArn":  
"arn:aws:wafv2:ap-northeast-1:127214194163:regional/webacl/leu-waf/c495e39f-c649-4115-81  
e5-57ae844b774a",  
    "LogDestinationConfigs": [  
      "arn:aws:logs:ap-northeast-1:127214194163:log-group:aws-waf-logs-leu-webacl01"  
    ],  
    "ManagedByFirewallManager": false,  
    "LogType": "WAF_LOGS",  
    "LogScope": "CUSTOMER"  
  }  
}
```

**Expected:** LogDestinationConfigs contains exactly one destination.

### B) Generate traffic (hits + blocks)

```
curl -I https://chewbacca-growl.com/  
curl -I https://app.chewbacca-growl.com/
```

### Code Block:

```
curl -I https://app.projectcomplete.me  
curl -I https://projectcomplete.me
```

### Result:

```
> curl -I https://app.projectcomplete.me  
curl -I https://projectcomplete.me  
HTTP/2 200  
date: Tue, 10 Feb 2026 02:30:09 GMT  
content-type: text/html; charset=utf-8  
content-length: 93  
server: Werkzeug/3.1.5 Python/3.9.25  
  
HTTP/2 200  
date: Tue, 10 Feb 2026 02:30:10 GMT  
content-type: text/html; charset=utf-8  
content-length: 93  
server: Werkzeug/3.1.5 Python/3.9.25
```

```
HTTP/2 200  
date: Tue, 10 Feb 2026 02:30:09 GMT  
content-type: text/html; charset=utf-8  
content-length: 93  
server: Werkzeug/3.1.5 Python/3.9.25
```

```
HTTP/2 200  
date: Tue, 10 Feb 2026 02:30:10 GMT  
content-type: text/html; charset=utf-8  
content-length: 93  
server: Werkzeug/3.1.5 Python/3.9.25
```

---

### C2) If S3 destination

```
aws s3 ls s3://aws-waf-logs-<project>-<account_id>/ --recursive | head
```

### Code Block:

## Armageddon Lab 1c\_bonus-D Student verification (CLI)

aws s3 ls

s3://aws-waf-logs-lew-127214194163/AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/ --recursive | head

### Result:

```
> aws s3 ls s3://aws-waf-logs-lew-127214194163/AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/ --recursive | head
2026-02-09 17:14:04      694 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/05/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0105Z_61520a44.log.gz
2026-02-09 17:09:04      972 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/05/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0105Z_f109ec94.log.gz
2026-02-09 17:24:05      612 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/15/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0115Z_b62e486a.log.gz
2026-02-09 17:24:05     1030 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/20/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0120Z_1fa92bb6.log.gz
2026-02-09 17:29:04      574 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/25/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0125Z_258817df.log.gz
2026-02-09 17:39:04      760 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/35/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0135Z_1be5720d.log.gz
2026-02-09 17:49:05      843 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/45/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0145Z_87da8403.log.gz
2026-02-09 18:04:04      757 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/55/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0155Z_60a70332.log.gz
2026-02-09 17:59:04      576 AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/55/127214194163_waflogs_ap-northeast-1_lew-waf_20260210T0155Z_7375f21e.log.gz
```

2026-02-09 17:14:04 694

AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/05/127214194163\_waflogs\_ap-northeast-1\_lew-waf\_20260210T0105Z\_61520a44.log.gz

2026-02-09 17:09:04 972

AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/05/127214194163\_waflogs\_ap-northeast-1\_lew-waf\_20260210T0105Z\_f109ec94.log.gz

2026-02-09 17:24:05 612

AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/15/127214194163\_waflogs\_ap-northeast-1\_lew-waf\_20260210T0115Z\_b62e486a.log.gz

2026-02-09 17:24:05 1030

AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/20/127214194163\_waflogs\_ap-northeast-1\_lew-waf\_20260210T0120Z\_1fa92bb6.log.gz

2026-02-09 17:29:04 574

AWSLogs/127214194163/WAFLogs/ap-northeast-1/lew-waf/2026/02/10/01/25/127214194163\_waflogs\_ap-northeast-1\_lew-waf\_20260210T0125Z\_258817df.log.gz

---

## 1. “Are 5xx caused by attackers or backend failure?”

You can differentiate these by comparing the **terminating action** in WAF logs against the **backend status codes** in ALB logs.

## Armageddon Lab 1c\_bonus-D Student verification (CLI)

- **Attack-driven:** If WAF logs show a spike in **BLOCK** actions for specific rules (e.g., SQLi, XSS) while the ALB shows **502** or **504** errors, the traffic volume of the attack may be overwhelming your integration.
- **Backend-driven:** If WAF actions are mostly **ALLOW**, but the ALB logs show a high **target\_status\_code** of **5xx**, your backend application is failing to process legitimate-looking requests.

### 2. “Do we see WAF blocks spike before ALB 5xx?”

This identifies a “**Pre-cursor Attack**” or a “**Collateral Damage**” scenario.

- Use **CloudWatch Contributor Insights** to overlay **BlockedRequests** from WAF with **HTTPCode\_Target\_5XX\_Count** from the ALB.
- If the WAF blocks spike *first*, you are likely seeing a volumetric attack (like a Layer 7 DDoS) where the WAF is working, but the sheer volume of “allowed” traffic or the overhead of processing the “blocked” traffic is causing the ALB or backend to throttle and fail.

### 3. “What paths / IPs are hammering the app?”

WAF logs provide granular visibility into the **httpRequest** object. You can run a **CloudWatch Logs Insights** query to find the culprits:

SQL

This reveals if the traffic is a “low and slow” attack on a specific expensive endpoint (like **/api/search**) or a broad scan across the entire site.

### 4. “Is it one client, one ASN, one country, or broad?”

Security events are often localized. Analyzing the **ASN (Autonomous System Number)** and **Country Code** helps determine the scope of a block.

- **One ASN:** Likely a specific ISP or cloud provider being used as a proxy/botnet.
- **One Country:** Potential geo-targeted attack; can be mitigated with a WAF Geographic Match rule.
- **Broad:** A distributed botnet or a genuine global traffic surge (e.g., a viral marketing event).

### 5. “Did WAF mitigate, or are we failing downstream?”



## Armageddon Lab 1c\_bonus-D Student verification (CLI)

This is the ultimate "effectiveness" check.

- **Successful Mitigation:** WAF logs show `action: BLOCK` and the ALB logs show the corresponding requests never reached the `target_processing_time`.
- **Downstream Failure:** If the WAF shows `action: ALLOW` (or if you are in `COUNT` mode) and the backend is returning `503 Service Unavailable`, your WAF rules are too permissive, and the "malicious" traffic is successfully reaching and breaking your servers.

lab 1c-E prompt

Directly from Theo's repo

Key update since "the old days": AWS WAF logging can go directly to CloudWatch Logs, S3, or Kinesis Data Firehose, and you can associate one destination per Web ACL. Also, the destination name must start with aws-waf-logs-.

Terraform supports this with `aws_wafv2_web_acl_logging_configuration`.  
Terraform Registry

Below is Lab 1C-Bonus-E (continued): WAF logging in Terraform (with toggles), plus verification commands.

1) Add variables (append to variables.tf)

```
variable "waf_log_destination" {  
  description = "Choose ONE destination per WebACL: cloudwatch | s3 | firehose"  
  type        = string  
  default     = "cloudwatch"  
}
```

```
variable "waf_log_retention_days" {  
  description = "Retention for WAF CloudWatch log group."  
  type        = number  
  default     = 14  
}
```

```
variable "enable_waf_sampled_requests_only" {  
  description = "If true, students can optionally filter/redact fields later. (Placeholder toggle.)"  
  type        = bool  
  default     = false  
}
```

2) Add file: `bonus_b_waf_logging.tf` (Look in Folder)

This provides three skeleton options (CloudWatch / S3 / Firehose). Students choose one via `var.waf_log_destination`.

3) Outputs (append to outputs.tf)

# Explanation: Coordinates for the WAF log destination—Chewbacca wants to know where the footprints landed.

```
output "chewbacca_waf_log_destination" {
  value = var.waf_log_destination
}

output "chewbacca_waf_cw_log_group_name" {
  value = var.waf_log_destination == "cloudwatch" ?
aws_cloudwatch_log_group.chewbacca_waf_log_group01[0].name : null
}

output "chewbacca_waf_logs_s3_bucket" {
  value = var.waf_log_destination == "s3" ?
aws_s3_bucket.chewbacca_waf_logs_bucket01[0].bucket : null
}

output "chewbacca_waf_firehose_name" {
  value = var.waf_log_destination == "firehose" ?
aws_kinesis_firehose_delivery_stream.chewbacca_waf_firehose01[0].name : null
}
```

#### 4) Student verification (CLI)

##### A) Confirm WAF logging is enabled (authoritative)

```
aws wafv2 get-logging-configuration \
  --resource-arn <WEB_ACL_ARN>
```

Expected: LogDestinationConfigs contains exactly one destination.

##### B) Generate traffic (hits + blocks)

```
curl -I https://chewbacca-growl.com/
curl -I https://app.chewbacca-growl.com/
```

##### C1) If CloudWatch Logs destination

```
aws logs describe-log-streams \
  --log-group-name aws-waf-logs-<project>-webacl01 \
  --order-by LastEventTime --descending
```

Then pull recent events:

```
aws logs filter-log-events \
  --log-group-name aws-waf-logs-<project>-webacl01 \
  --max-items 20
```

##### C2) If S3 destination

```
aws s3 ls s3://aws-waf-logs-<project>-<account_id>/ --recursive | head
```

C3) If Firehose destination

```
aws firehose describe-delivery-stream \  
--delivery-stream-name aws-waf-logs-<project>-firehose01 \  
--query "DeliveryStreamDescription.DeliveryStreamStatus"
```

And confirm objects land:

```
aws s3 ls s3://<firehose_dest_bucket>/waf-logs/ --recursive | head
```

5) Why this makes incident response “real”

Now you can answer questions like:

“Are 5xx caused by attackers or backend failure?”

“Do we see WAF blocks spike before ALB 5xx?”

“What paths / IPs are hammering the app?”

“Is it one client, one ASN, one country, or broad?”

“Did WAF mitigate, or are we failing downstream?”

This is precisely why WAF logging destinations include CloudWatch Logs (fast search) and S3/Firehose (archive/SIEM pipeline)